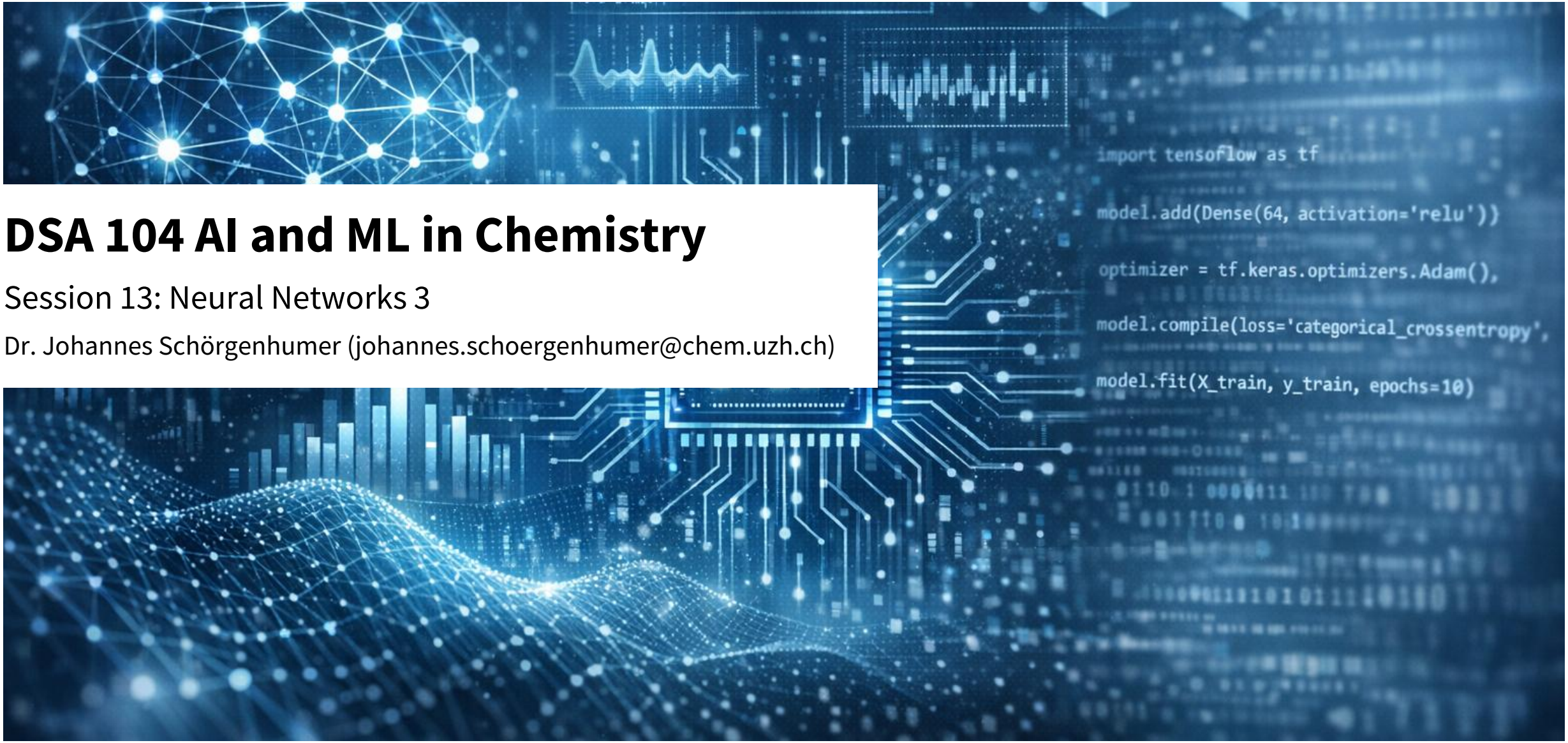
Universität Zürich[UZH]

Department of Chemistry



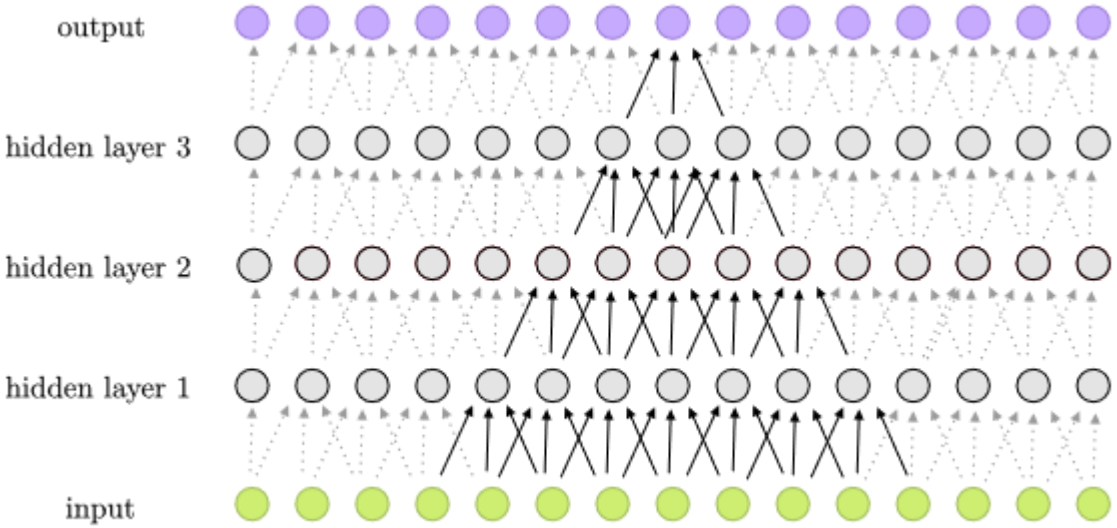
DSA 104 AI and ML in Chemistry

Session 13: Neural Networks 3

Dr. Johannes Schörgenhumer (johannes.schoergenhumer@chem.uzh.ch)

Types of Neural Networks

- **FNN/MLP (Feedforward NN)**
- **CNN (Convolutional NN)**
- RNN (Recurrent NN)
- Transformers
- Encoders
- GAN (Generative Adversarial N)
- **GNN (Graph NN)**
- RL networks (Reinforcement Learning N)
- Diffusion models



CNN illustration by Vicente Oyanedel M. - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=95810663
Basic building blocks for GNN by NickDiCicco - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=119932650

Why CNNs?

— Tabular data is easy to handle ☺

— What about images?

  — Example: 250 x 250 pixels x 3 color channels = 187.5k input dimensions

Plenty of uses in life sciences, e.g.

— Microscopy (cells, tissues) → Cell classification

— Medical imaging (MRI, CT) → Desease detection

— Structural biology (protein structures) → Protein structure analysis

Need for **automated feature extraction** → CNN

CNNs shine in **image** and audio learning, as well as for segmentation!

Track record in ImageNet Large Scale Visual Recognition Competition (ILSVRC – 1.2 M images, 1000 classes):

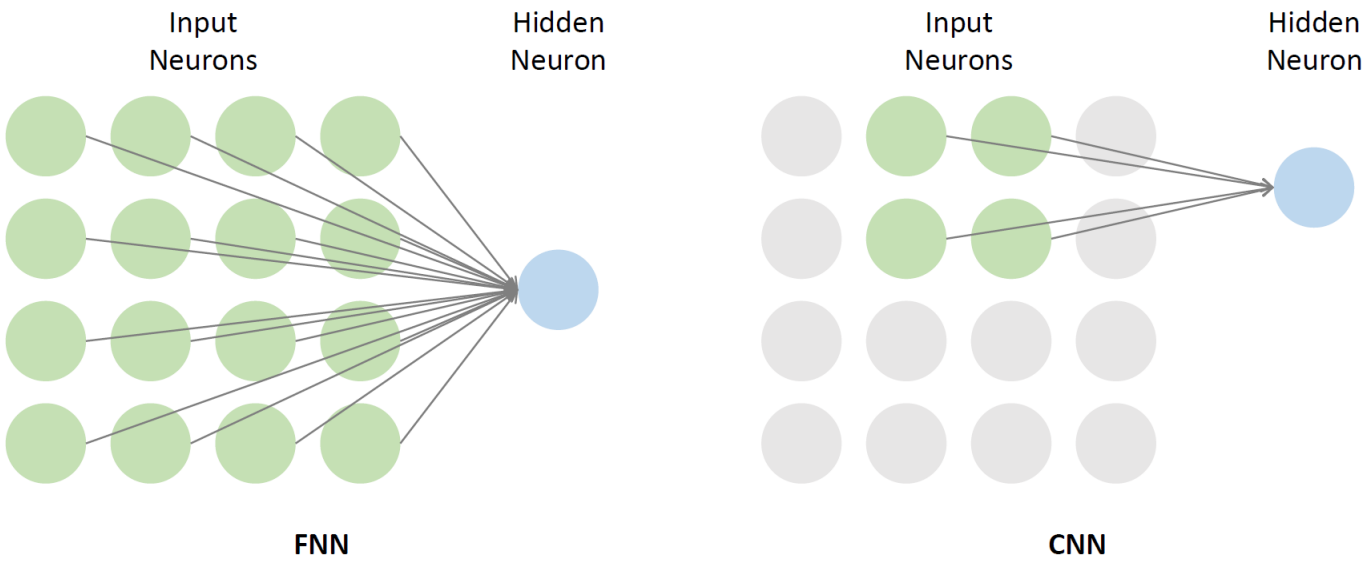
— ILSVRC 2012: Won by the only CNN-based solution.

— ILSVRC 2013: Best 5 participants were all CNNs (9 of the top 10 were CNNs).

— ILSVRC 2014: Everyone uses CNNs.

CNNs – Key principles: Receptive field

- Pixels that are near each other are **highly correlated**.
- Same basic patches (e.g., edges, corners) appear on all positions of the image.
- Detect those basic patches by only viewing a small part of the image.
  - Efficiency: Use a network with a small receptive field!
  - I.e. a neuron only sees a small part of the image
- **Receptive field**: Connect network to patch of image using a weight matrix (=**kernel** or **filter**)



Picture from A. Schörgenhumer, *Hands-on AI I*, Lecture materials, **2023**, JKU Linz.

CNNs – Key principles: Convolution

- Mathematically: Operation on two functions, e.g. in a 2D case $$(h * k)(a, b) = \sum_i \sum_j h(a + i, b + j)k(i, j)$$
- Technically: Cross-correlation or sliding dot product
- By convention: **Convolution**.

- Best illustration: Applying a **kernel** to the input as a **filter**
  - Each time the kernel is applied, an output value is created
  - To these outputs, an activation function is added (ReLU!)
  - **Kernel application + activation function = convolution layer**

CNNs – Key principles: Weight sharing and kernels

Same basic patches (e.g., edges, corners) appear on all positions of the image.

— Often, **invariances to certain variations** are desired (e.g., translation invariance, i.e. pattern detected regardless of position).

— **Receptive field** can be **reused** at all positions of the image to produce the new output (=feature/activation map), applying the **same weights**!

— Reusing the kernel weight matrix is called **weight sharing**.

— **Significantly reduces** number of model **parameters**.

E.g. k: 3x3, i: 6x6, o: 4x4

kernel

0	0	0.5
0	0.5	0
0	0	0.5

input

0	0	0	0	0	0
0	1.0	0	0	0.4	0
0	0	0	0.4	0	0
0	0	0	0	0.4	0
0	0	0	0.4	0	0
0	0	0	0	0.4	0

output

0.5	0.2	0	0.2
0	0	0.6	0
0	0.4	0	0.2
0	0	0.6	0

Picture from A. Schörgenhumer, *Hands-on AI I*, Lecture materials, **2023**, JKU Linz.

Interactive kernel demo

Have a look at here: https://setosa.io/ev/image-kernels/

Discuss what effect the kernel has.

Inspect different kernels. How would a kernel look for detecting vertical edges?

How are the corners and borders of the image handled?

CNNs – Key principles: Kernels

Convolution can be applied using existing kernels (E.g. edge detection (Sobel right))

But:

— Kernels are just small weight matrices W, they can be learned the same way as in regular neural networks.

— **Learn the kernels** ourselves!

Idea for CNNs:

— learn useful kernels, e.g., to detect edges, corners, color patches, body parts, . . .

— Gain global understanding by combining local patterns (build **hierarchy**!)

$$\rightarrow \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \rightarrow$$

Picture from A. Schörgenhumer, *Hands-on AI I*, Lecture materials, **2023**, JKU Linz.

CNNs – Key principles: Padding

Applying a kernel of size >1, output will be smaller than input

Padding keeps the input and output **sizes the same**!

Can be applied to the input layer or between any layer.

Types:

— Zero padding:

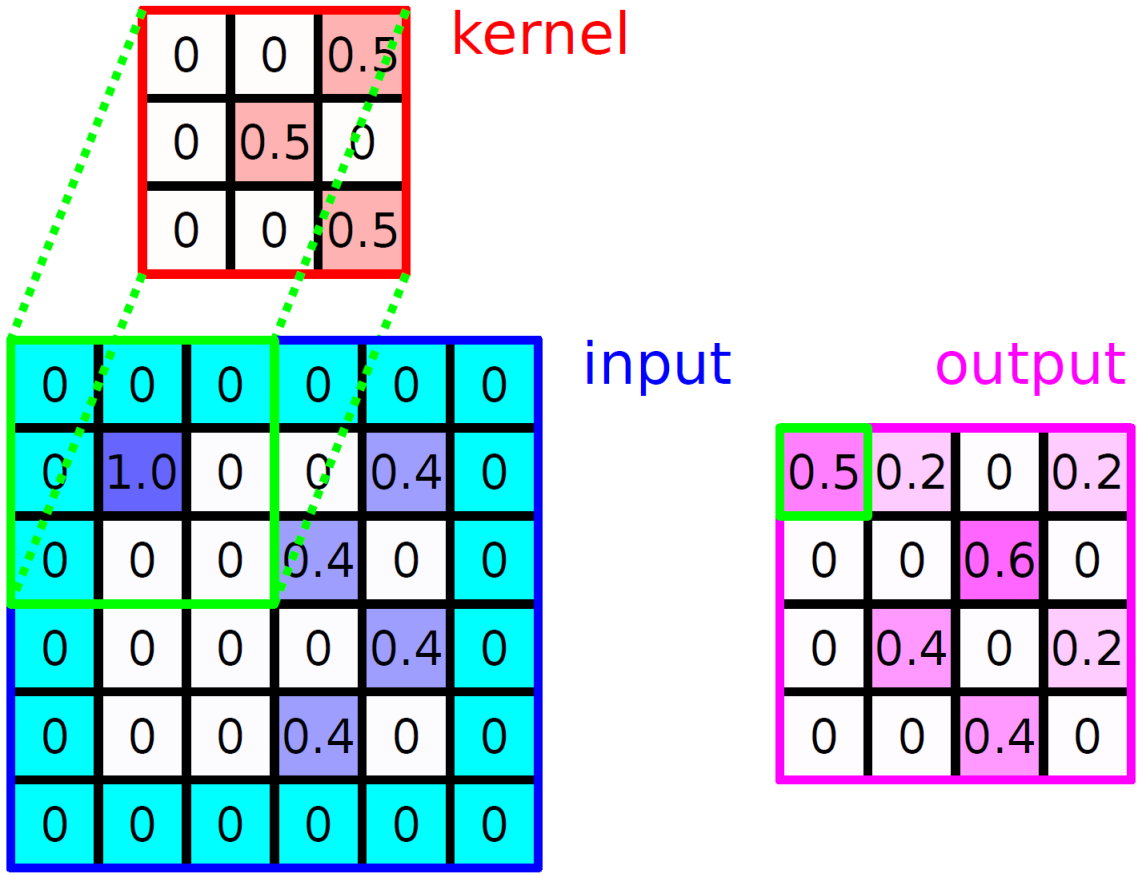
  — Most common

  — Adds zeros around the input

— Repeat adding: Duplicates the border value

— Others: mean, weighted sum, …

— E.g. (k: 3x3, i:4x4, o: 2x2) vs. (k: 3x3, i: (4+1)x(4+1), o: 4x4)



Picture from A. Schörgenhumer, *Hands-on AI I*, Lecture materials, **2023**, JKU Linz.

CNNs – Key principles: Striding

Striding controls how much the kernels/filters are moved. The smaller the stride, the more overlap of receptive filters

Striding is one way of **downsampling** images.

A stride > 1 will…

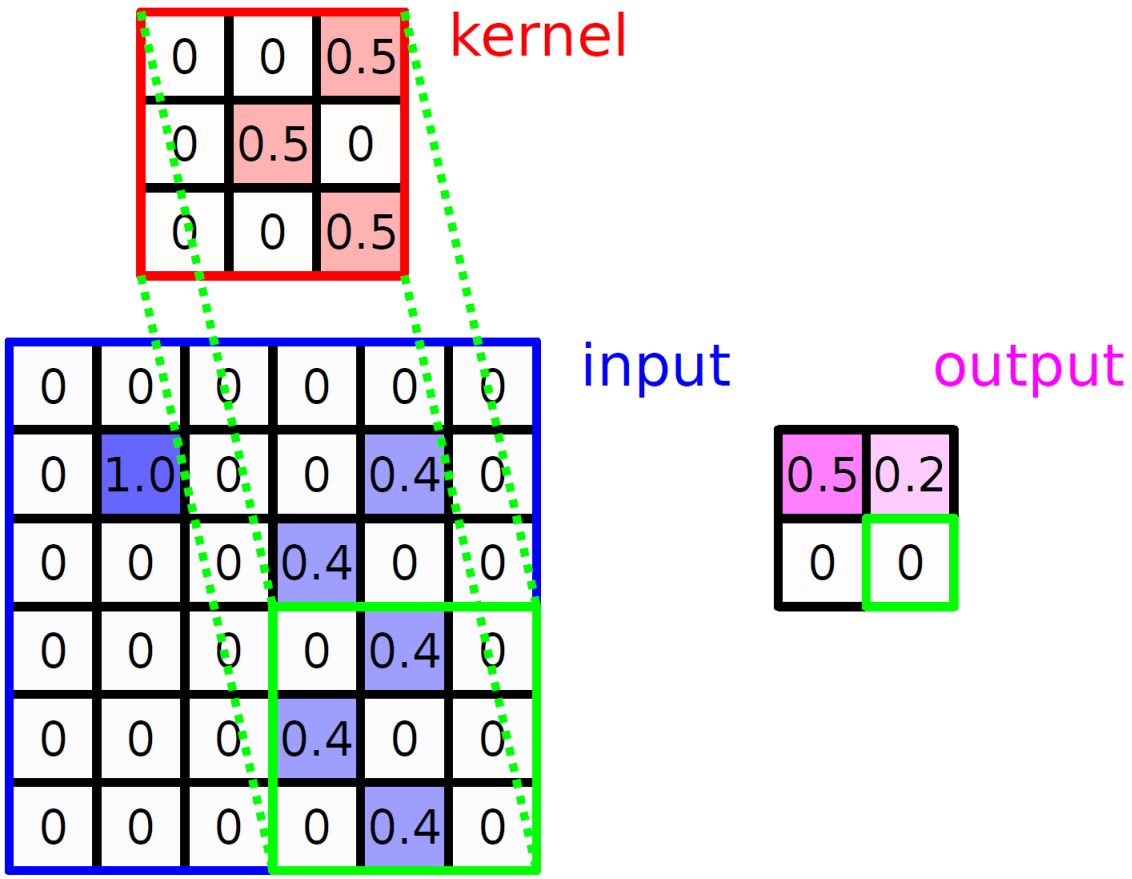
— Cause deeper neurons to “skip” parts of the input

— Change effective receptive field density, not its size (kernel stays the same!)

— Lead to **loss of information** (keep essential information!)

— Reduce computational load and memory requirements

— E.g. stride = 3



Picture from A. Schörgenhumer, *Hands-on AI I*, Lecture materials, **2023**, JKU Linz.

CNNs – Key principles: Pooling layers

Another way of downsampling: Pooling!

Purpose

— Reduce spatial size

— Keep important features

— Improve robustness (model ignores small positional changes)

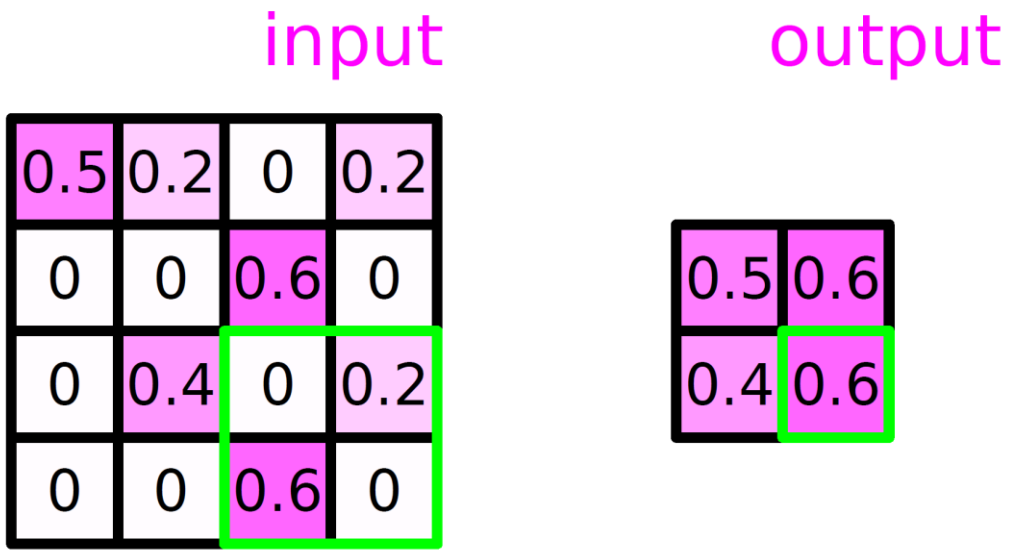
Types

— Max pooling (most common): take the highest value in a *k* x *k* field

— Average pooling

E.g. pooling size = 2: From a 4×4 region → 2×2 by taking max values

Pooling is a fixed operation, no parameters to learn (as compared to striding)



Picture from A. Schörgenhumer, *Hands-on AI I*, Lecture materials, **2023**, JKU Linz.

CNNs – Key principles: Inputs

Until now: Grayscale images, only 1 channel

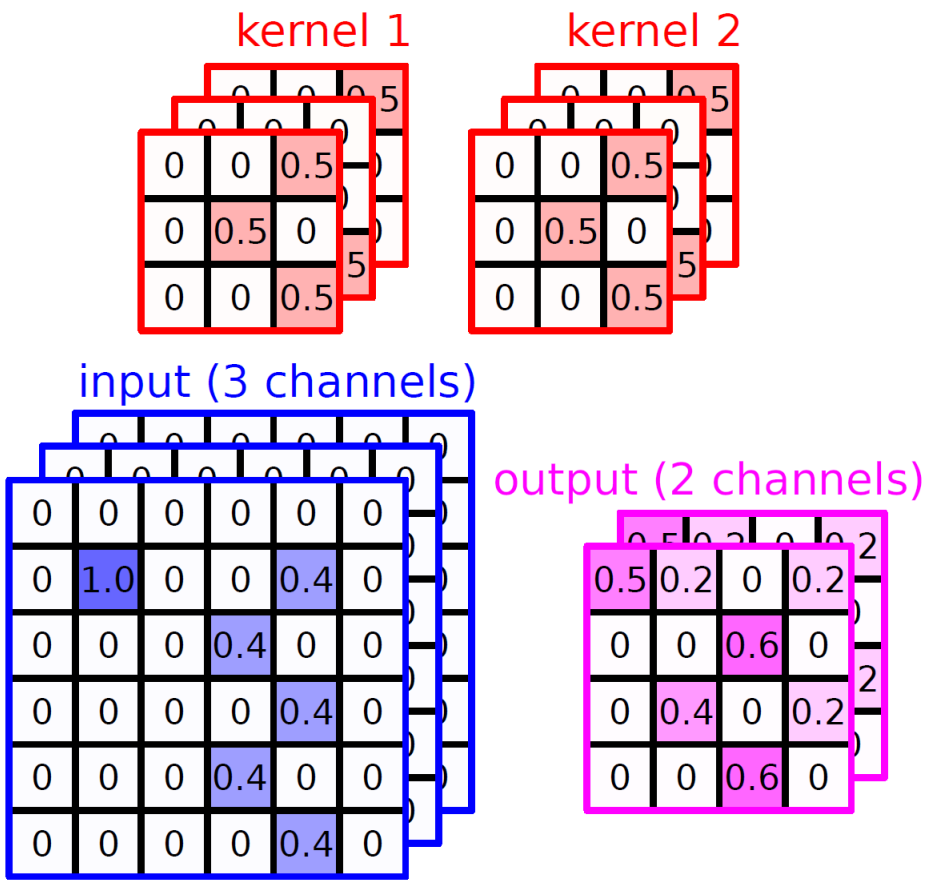
But: RGB images have 3 **channels** for red, green, blue, typically stored in a shape of (width, height, 3).

After the convolutional operations, channels are also called **feature maps** or **activation maps**.

Kernel needs to **match** the number of **channels** (e.g., if the input is 3D, the kernel must be 3D as well).

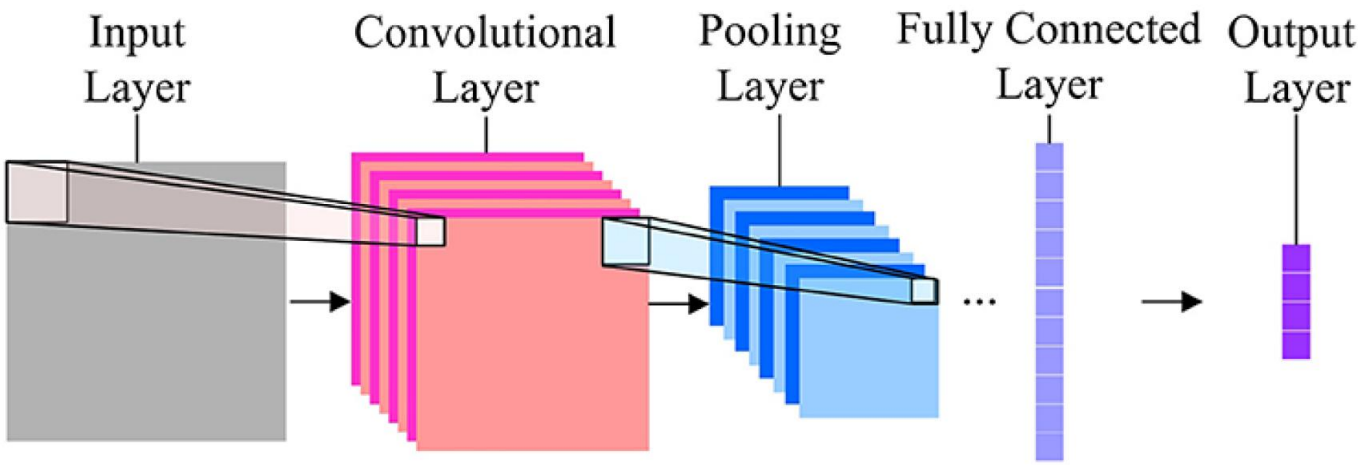
Regardless of the number of channels, each kernel produces a single feature map/channel.

Multiple kernels produce multiple outputs!



Picture from A. Schörgenhumer, *Hands-on AI I*, Lecture materials, **2023**, JKU Linz.

CNNs – Putting it all together



Typical flow:

1. **Input image** (e.g., microscopy image)
2. **Convolution layer** (detect edges, textures)
3. **Activation** (e.g. ReLU, keep strong signals)
4. **Pooling** (compress + stabilise)
5. Multiple layers of convolution, activation and (optional) pooling
6. **Flatten** multidimensional output into vector
7. **Fully connected** layer (classification)

The **complexity of detected features** tends to **increase** layer by layer!

Interactive visualisation and great summary: https://poloclub.github.io/cnn-explainer/

Image from Min Peng et al., *Front. Psychol.* **2017**, *8*, 1745. doi: 10.3389/fpsyg.2017.01745

How CNNs work – Biology as example for hierarchy

— Complexity increases layer by layer

— Global understanding built from local patterns

— Naturally build a **hierarchy** of features!

— E.g. Cell imaging in biology:

Layer	What it learns	Biology example
Early	edges	cell boundaries
Mid	shapes	nuclei
Deep	structures	abnormal cells
Final	semantics	disease classification

CNNs – Limitations

CNNs excel at local feature extraction, but…

— …struggle with long-range dependencies

— …need huge sets of labelled data

— …often require a lot of memory (feature maps) and processing power (GPU / TPU!)

— …may be sensitive towards distortions and geometric distortions

— …may struggle with out-of-distribution generalisation (e.g. fail with unknown backgrounds, textures, lighting)

— …may rely more on textures than on shapes

— …show very limited interpretability

CNN Application Examples

— Cell classification (healthy vs. diseased)

— Cell segmentation (nuclei detection)

— Tumor detection in histology slides

— Protein contact map prediction

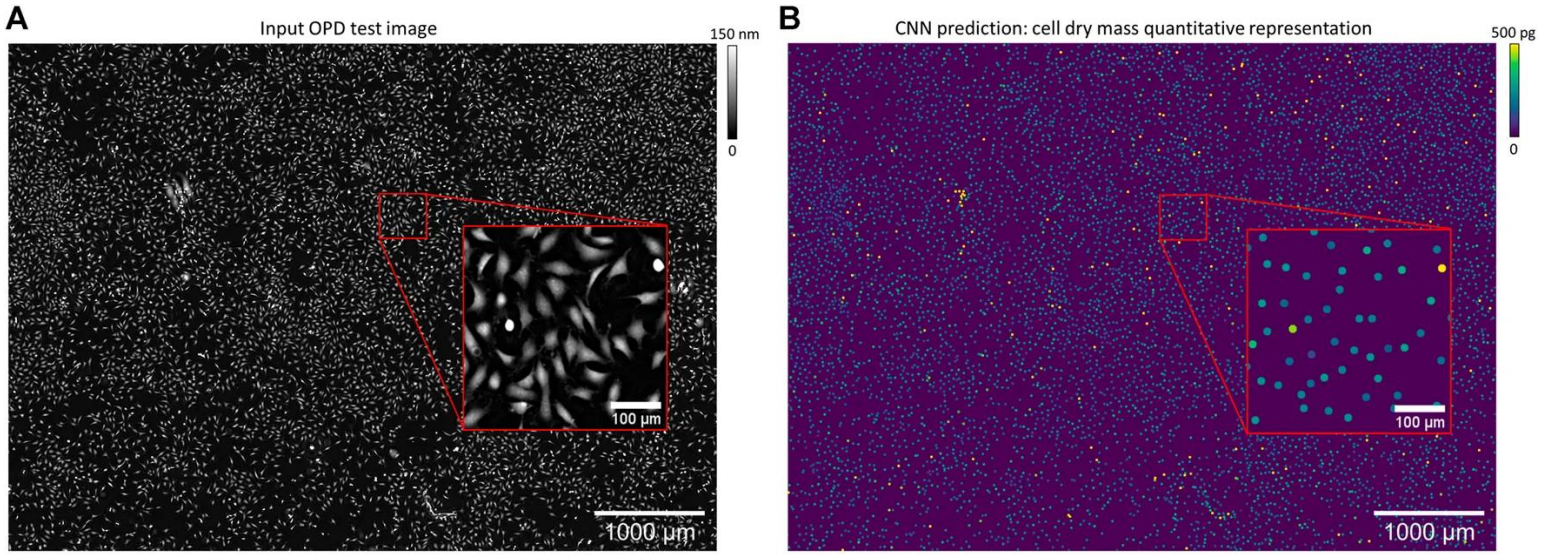
— Structure classification

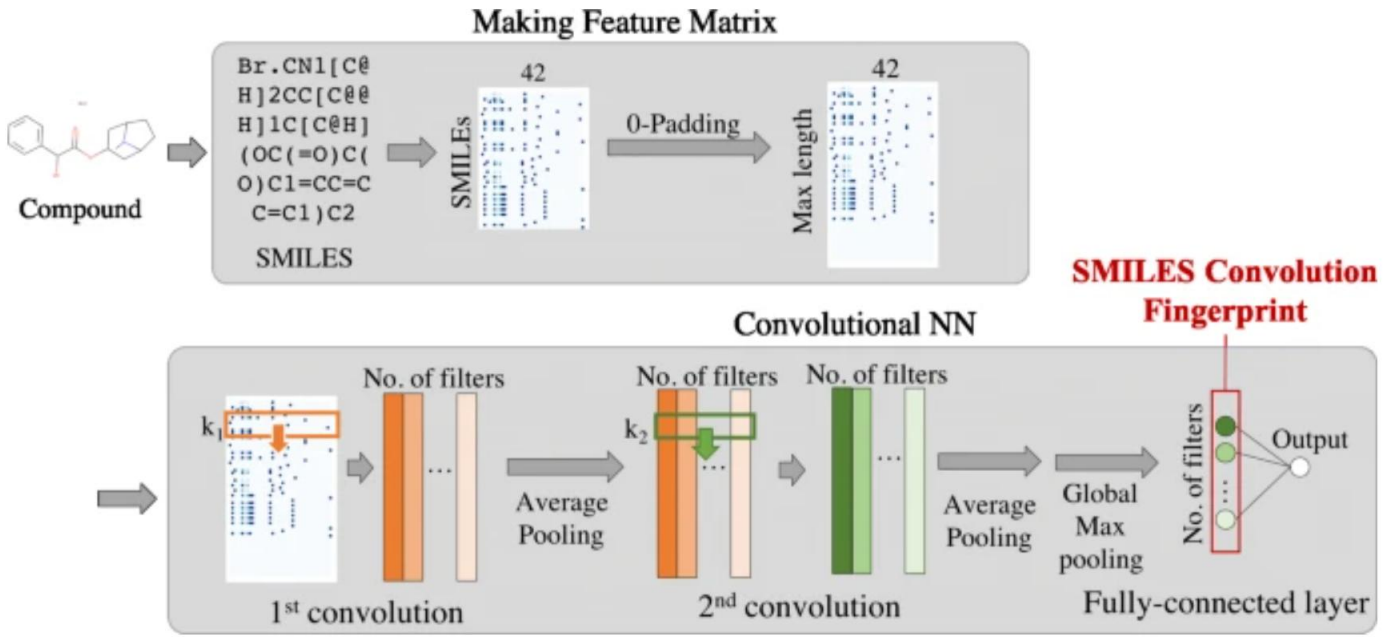
— Spectra as images (e.g. IR, NMR)

— Reaction prediction (e.g. smiles encoding)

— Process parameters (encoded as 2D image, …)

.

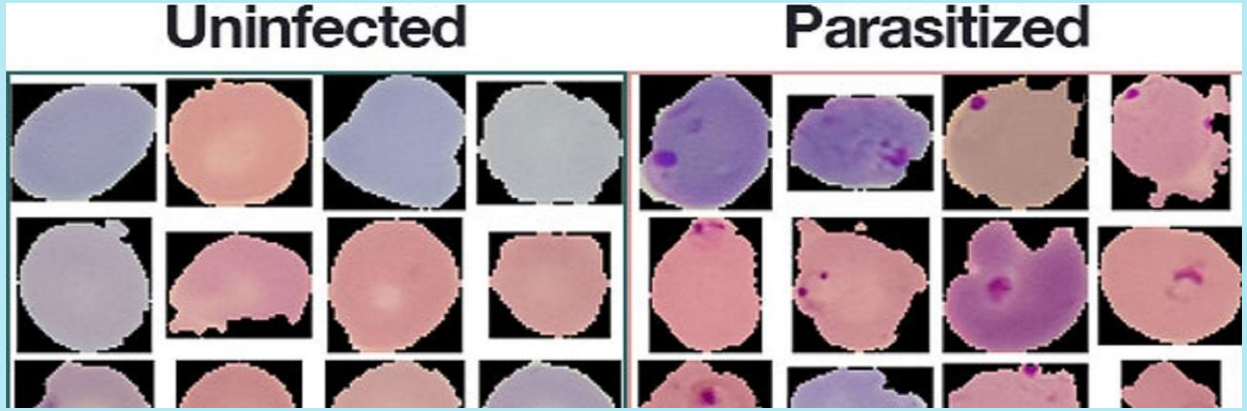
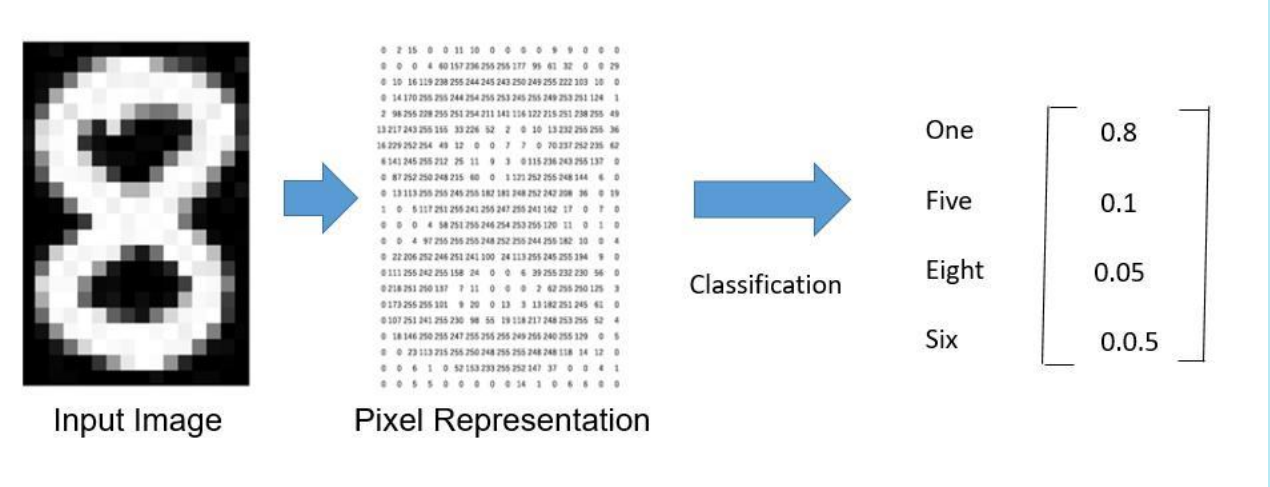




Allier C. et al., Front. Phys. **2022**, *9*, 776805. doi: 10.3389/fphy.2021.776805
Hirohara, M. et al., *BMC Bioinformatics* **2018**, *19*, 526. https://doi.org/10.1186/s12859-018-2523-5

CNN Exercises

1) **Demo:** Cell Classification with CNN (Malaria Cell Image Dataset)



2) **Exercise1**: Build a small CNN to classify handwritten numbers (MNIST dataset – very clean)